Q1) Write a SQL query to display the names and ratings of restaurants (from a table named Restaurants) where the rating is higher than the average rating of all restaurants in the table.

Hint: Use a subquery in the WHERE clause to calculate the average rating.

```sql
SELECT name, rating
FROM Restaurants
WHERE rating > (
    SELECT AVG(rating)
    FROM Restaurants
);
```

Q2) In a Flipkart-style Products table (columns: product_id, name, price, category), write a SQL query to list each product name along with the average price of its category as an additional column using a scalar subquery in the SELECT statement.

```sql
SELECT
    name,
    price,
    category,
    (SELECT AVG(p2.price)
     FROM Products p2
     WHERE p2.category =
p1.category) AS
average_category_price
FROM Products p1;
```

Q3) Given a Playlists table (playlist_id, user_id, playlist_name) and a Users table (user_id, username), write a SQL query using a subquery in the FROM clause to show each username and the number of playlists they have created, displaying only users who have created more playlists than the average number of playlists per user.

Hint: Use a derived table (subquery in FROM) to count playlists per user, then filter with a subquery in WHERE.

```sql
SELECT u.username,
p.playlist_count
FROM Users u
JOIN (
    SELECT user_id,
COUNT(*) AS
playlist_count
    FROM Playlists
    GROUP BY user_id
) p ON u.user_id =
p.user_id
WHERE p.playlist_count >
(
    SELECT
AVG(user_playlist_count)
    FROM (
        SELECT COUNT(*)
AS user_playlist_count
        FROM Playlists
        GROUP BY user_id
    ) AS avg_playlists
);
```

Q4) Suppose you have an Orders table (order_id, user_id, total_amount) for a food delivery app. Write a query to find all user_ids who have placed at least one order with a total_amount greater than the average order amount, using a subquery in the WHERE clause.

```sql
SELECT DISTINCT user_id
FROM Orders
WHERE total_amount > (
    SELECT
AVG(total_amount)
    FROM Orders
);
```